Capstone 08 – Day 1 Reference Document

Discovery, Architecture and Ingestion (Bronze layer)

This is the master reference for Day 1. Days 2–5 depend on what is built and named here, so every table name, column name, path and decision below is deliberate. Follow the names exactly. If you must change one, change it in this document first and log it in the decision log.

How later documents refer to this one: conventions are `C-xx`, steps are `S1` – `S9`, deliverables are `D1-xx`, decisions are `DEC-xx`, risks/assumptions are `R-xx` / `A-xx`, tests are `TC..` (from the project worksheet) or `D1-T..` (Day 1 extras), and hand-off items for Day 2 are `X-xx`.

---

1. Day 1 at a glance

Objective: understand the brief, design the solution, profile all source data, build the platform foundation (Git, Unity Catalog schema, external volume) and land all five sources in append-only Bronze Delta tables using restart-safe Auto Loader — with lineage and an audit proving nothing was lost or changed.

Day 1 is done when:

1. Five Bronze tables are loaded and reconciled to the source files.
2. Re-running ingestion adds **zero** rows.
3. The schema change in batches 10–12 is handled and evidenced.
4. Design documents, data dictionary, source profile, decision log and RAID tracker are in Git.
5. Day 1 tests pass and are logged.
6. A pull request is open with the PR checklist completed.

Time plan (8 hours)

Block	Time	Step	Main outputs
1	40 min	S1 Kickoff, BRS summary, logs	D1-01, D1-02, D1-03
2	40 min	S2 Git repo and branches	D1-04

Block	Time	Step	Main outputs
3	55 min	S3 Unity Catalog schema, external volume, file staging	D1-05, D1-06
4	70 min	S4 Source profiling	D1-07
5	70 min	S5 HLD, LLD, ERD, mapping, data dictionary	D1-08 to D1-12
6	100 min	S6 Build Bronze with Auto Loader	D1-13, D1-14
7	35 min	S7 Bronze audit and lineage	D1-15, D1-16
8	50 min	S8 Day 1 tests	D1-17
9	20 min	S9 Commit, PR, end-of-day log	D1-18

---

2. Conventions registry (do not deviate)

ID	Item	Value
C-01	Catalog	`<catalog>` – your workspace catalog. Set once through the `catalog` widget in `00_config`. Never hard-code it elsewhere.
C-02	Schema	`hospital_ops` (one schema; the layer is shown by the table-name prefix)
C-03	External volume	`raw_landing`; base path `/Volumes/<catalog>/hospital_ops/raw_landing` (called `BASE` in code)
C-04	Landing folders under BASE	`primary_events/`, `status_updates/`, `secondary_events/`, `reference/department_id/`, `reference/patient_id/` (files "arrive" here; Auto Loader reads only these)
C-05	Archive folder	`source_archive/` mirrors C-04 and holds the untouched original files. Auto Loader never reads it.

ID	Item	Value
C-06	Meta folders	`_meta/schemas/<stream>/` and `_meta/checkpoints/<stream>/` (outside every source folder)
C-07	Stream names	`encounters`, `status_cdc`, `secondary`, `department_id`, `patient_id`
C-08	Bronze tables	`bronze_encounter_events`, `bronze_status_cdc`, `bronze_secondary_events`, `bronze_department_id`, `bronze_patient_id`
C-09	Ops tables	`ops_bronze_audit`, `ops_test_results` (these will be reused on Days 3–5)
C-10	Lineage columns on every Bronze table	`_source_file`, `_source_file_name`, `_source_file_mod_time`, `_source_row_number`, `_ingested_at`, `_batch_id`, `_source_mode` (Auto Loader may also add `_rescued_data`; keep it)
C-11	Bronze data types	**All source columns stay STRING.** Typing happens in Silver. This preserves raw values exactly.
C-12	Repo and branches	Repo `hospital-ops-lakehouse`; branches `feature/day1-docs` and `feature/day1-bronze`; `main` is protected
C-13	Notebook files	`00_config`, `01_profile_sources`, `02_bronze_lib`, `03_bronze_run`, `04_bronze_audit`, `tests/test_day1_bronze`
C-14	Commit prefix	`D1:` followed by a short sentence, e.g. `D1: add bronze auto loader for encounters`

3. Prerequisites checklist (before you start)

Tick each one. If something is missing, ask your admin **now** and write it in the RAID tracker.

- A Databricks workspace with **Unity Catalog** enabled
- Compute: serverless, or a cluster in standard/single-user access mode that supports Unity Catalog
- On your catalog: `USE CATALOG` and `CREATE SCHEMA`

- An **existing external location** (a registered ADLS Gen2 path) and the privilege `CREATE EXTERNAL VOLUME` on it
- A Git provider account (GitHub, Azure DevOps, GitLab) and a personal access token linked in Databricks
- The project zip extracted on your laptop (you need `datasets/` and `technical_reference/`)
- Optional: Databricks CLI installed and authenticated

If you cannot get `CREATE EXTERNAL VOLUME`: create a managed volume instead (`CREATE VOLUME hospital_ops.raw_landing`), keep every other name identical, and record it as a deviation (DEC-13) and a risk. Everything downstream still works.

S1. Kickoff, BRS summary and logs (40 min)

S1.1 Write the BRS summary (D1-01)

Create `docs/01_brs_summary.md` with these sections. The content below is already extracted from the BRD – copy it and put it in your own words.

- **Business problem:** Encounter events and reference records arrive fragmented. Timestamps, status revisions and business keys are handled inconsistently, so reporting is late and exceptions cannot be explained or traced.
- **Objective:** Reduce waiting and discharge delays; support patient flow and capacity planning.
- **KPIs (Gold, Day 3):** waiting time, length of stay, bed utilization.
- **Stakeholders:** operations manager, analytics lead, data governance owner, platform operations, client product owner.
- **Functional requirements:** ingest events; process status CDC; publish validated Gold marts.
- **Non-functional requirements:** restart-safe pipelines, least-privilege access, traceable source metadata, freshness reporting, reproducible jobs, failure alerts.
- **Acceptance conditions:** expected-vs-actual reconciliation; no duplicate business keys in curated current-state tables; documented exceptions; repeatable end-to-end workflow; client walkthrough.
- **Out of scope:** live transaction decisions, production model serving, enterprise CI/CD, real patient PII, external notification integrations.

- **Source grain:** *one event row per `encounter_id` per source arrival.* In plain words: the same encounter may legitimately appear in more than one file; each appearance is one row in Bronze.
- **Platform boundary:** ADLS Gen2 + Auto Loader → Unity Catalog external volume → Bronze → Silver → Gold → SQL Warehouse / dashboards / Genie, orchestrated by Workflows, code in Git.

S1.2 Start the decision log (D1-02)

Create `docs/decision_log.md`. One row per decision: ID, date, decision, reason, alternatives rejected, owner. Seed it with these:

ID	Decision	Reason
DEC-01	Source grain = one event row per `encounter_id` per source arrival	Stated in the project guide; keeps raw history
DEC-02	One schema `hospital_ops`; layer shown by table prefix	Guide asks for one UC schema; BRD table names already carry the prefix
DEC-03	Azure SQL is simulated with the two CSV extracts unless a live JDBC read is actually run	BRD forbids presenting an extract as a live connection
DEC-04	Bronze stores every column as STRING	Keeps raw values for traceability; avoids load failures from bad values
DEC-05	Auto Loader `schemaEvolutionMode = addNewColumns`; `source_schema_version` is an **allowed** addition from batch 10 onward	Controlled schema change in the source contract
DEC-06	Reference CSVs also load through Auto Loader	Same checkpoint safety; re-runs cannot duplicate rows
DEC-07	Originals are archived in `source_archive/` and "released" to landing folders in waves	Lets you prove incremental loading without ever editing a source file

Add these as **OPEN** now (to be decided on Day 2/3 – they are findings from Day 1 profiling, see S4):

ID	Open question	Needed by
DEC-08	Deduplication tie-breaker (all 40 duplicates are identical on the 12 base columns, so `ingest_timestamp` cannot break ties)	Day 2
DEC-09	Outlier rule for `bill_amount` (see S4: a rule that isolates exactly 35)	Day 2
DEC-10	Mapping of statuses to admission / transfer / discharge	Day 2
DEC-11	Handling of the `priority` column that holds only "USD"	Day 2
DEC-12	Bed-capacity assumption for bed utilization (no capacity data exists)	Day 3

S1.3 Start the RAID tracker (D1-03)

Create `docs/raid.md` with columns: ID, type (Risk / Assumption / Issue / Dependency), description, impact, mitigation, owner, status. Seed it:

ID	Type	Description	Mitigation
R-01	Risk	Azure SQL not live; reference data is a CSV simulation	Label everywhere as simulation (DEC-03)
R-02	Risk	No bed capacity or admit/discharge timestamps in the data	Document derived definitions and assumptions (DEC-10, DEC-12)
R-03	Risk	Status words in the data do not match "admission / transfer / discharge"	Define and get approval for a mapping (DEC-10)
R-04	Risk	The status-update file has no late or out-of-order records	Author small test files on Day 2 for TC04
R-05	Issue	First Auto Loader run after batch 10 will fail once because of the new column (expected)	Re-run; Day 4 job retry handles it automatically

ID	Type	Description	Mitigation
A-01	Assumption	Data is synthetic; no real PII	Still apply masking practice on Day 2
D-01	Dependency	Admin grants for external volume and Git token	Raise in prerequisites

---

S2. Git repository and branches (40 min)

S2.1 Create the repo and skeleton

In your Git provider create an empty repo `hospital-ops-lakehouse` and protect `main` (require a pull request). Then on your laptop:

```
git clone <repo-url>
cd hospital-ops-lakehouse
mkdir -p docs notebooks sql tests config evidence
touch docs/.gitkeep notebooks/.gitkeep sql/.gitkeep tests/.gitkeep
config/.gitkeep evidence/.gitkeep
git checkout -b feature/day1-docs
```

Create a `.gitignore` containing at least: `.env`, `*.token`, `.DS_Store`, `__pycache__/`, `*.zip`. **Never commit tokens, keys or the raw dataset zip.**

Create a short `README.md` (project name, one-paragraph purpose, folder map, how to run Day 1). Create `docs/PR_CHECKLIST.md`:

```
## PR checklist
- [ ] Notebooks run top to bottom without errors
- [ ] Tests run and results saved (evidence/)
- [ ] No secrets, tokens or hard-coded catalog names
- [ ] decision_log.md and raid.md updated
- [ ] Counts reconcile (audit table shows no FAIL)
- [ ] Bronze data was not modified or deleted
- [ ] Reviewer named
```

Commit and push: `git add . && git commit -m "D1: repo skeleton, README, PR checklist" && git push -u origin feature/day1-docs`.

S2.2 Connect Databricks to the repo

1. Databricks → **Settings → Linked accounts** → add your Git provider and personal access token.
2. **Workspace → Create → Git folder** → paste the repo URL → Create.
3. In the Git folder, use the branch dropdown to switch to `feature/day1-docs`. Later create `feature/day1-bronze` from `main` the same way.

Notebooks stored in Git are plain `.py` files. Each begins with `# Databricks notebook source` and cells are separated by `# COMMAND ----------`. Databricks shows them as normal notebooks.

Rule for the whole project: all work happens on a feature branch; `main` changes only through a reviewed PR (D1-04).

S3. Unity Catalog schema, external volume and file staging (55 min)

S3.1 Create the schema and volume

Run in a SQL editor or notebook SQL cell. Replace the placeholders.

```
USE CATALOG <catalog>;
CREATE SCHEMA IF NOT EXISTS hospital_ops COMMENT 'Capstone 08 - Hospital
Patient Journey and Operations';

-- find the existing external location and its storage URL
SHOW EXTERNAL LOCATIONS;
DESCRIBE EXTERNAL LOCATION <your_external_location>;

-- the volume path must sit inside that external location
CREATE EXTERNAL VOLUME IF NOT EXISTS hospital_ops.raw_landing
  LOCATION
'abfss://<container>@<storageaccount>.dfs.core.windows.net/<path>/hospital_op
s'
  COMMENT 'Landing zone and archive for Capstone 08 raw files';

DESCRIBE VOLUME hospital_ops.raw_landing;
```

Plain English: the schema is a folder for tables; the external volume is a folder for *files* that lives in your own cloud storage but is governed by Unity Catalog, so permissions and lineage apply to it.

S3.2 Create the shared config notebook (D1-14, part 1)

Create `notebooks/00_config.py`. Every other notebook starts with `%run ./00_config`.

```
# Databricks notebook source
# 00_config - shared constants (conventions C-01 to C-10)
dbutils.widgets.text('catalog', '<catalog>')
CATALOG = dbutils.widgets.get('catalog')
SCHEMA = 'hospital_ops'
VOLUME = 'raw_landing'
FQ = f'{CATALOG}.{SCHEMA}'
BASE = f'/Volumes/{CATALOG}/{SCHEMA}/{VOLUME}'
ARCHIVE = f'{BASE}/source_archive'
META = f'{BASE}/_meta'

spark.sql(f'USE CATALOG {CATALOG}')
spark.sql(f'USE SCHEMA {SCHEMA}')

# landing sub-folders (relative to BASE)
FOLDERS = ['primary_events', 'status_updates', 'secondary_events',
           'reference/department_id', 'reference/patient_id']

STREAMS = {
  'encounters':    {'sub': 'primary_events',            'target':
f'{FQ}.bronze_encounter_events', 'mode': 'ADLS_AUTOLOADER'},
  'status_cdc':    {'sub': 'status_updates',            'target':
f'{FQ}.bronze_status_cdc',       'mode': 'ADLS_AUTOLOADER'},
  'secondary':     {'sub': 'secondary_events',          'target':
f'{FQ}.bronze_secondary_events', 'mode': 'ADLS_AUTOLOADER'},
  'department_id': {'sub': 'reference/department_id',   'target':
f'{FQ}.bronze_department_id',    'mode': 'CSV_EXTRACT_SIMULATION'},
  'patient_id':    {'sub': 'reference/patient_id',      'target':
f'{FQ}.bronze_patient_id',       'mode': 'CSV_EXTRACT_SIMULATION'},
}

REQUIRED_PRIMARY = ['encounter_id', 'event_timestamp', 'ingest_timestamp',
'department_id',
                    'patient_id', 'visit_type', 'encounter_status',
```

```
'priority',
                    'bill_amount', 'wait_minutes', 'doctor_id',
'source_system']
OPTIONAL_PRIMARY = ['source_schema_version']          # allowed from batch 10
(DEC-05)
LINEAGE_COLS = ['_source_file', '_source_file_name', '_source_file_mod_time',
                '_source_row_number', '_ingested_at', '_batch_id',
'_source_mode', '_rescued_data']

def release(sub, names):
    '''Copy original files from the archive into the landing folder
(simulates arrival).'''
    for n in names:
        dbutils.fs.cp(f'{ARCHIVE}/{sub}/{n}', f'{BASE}/{sub}/{n}')
        print('released', sub, n)
```

S3.3 Create folders and stage the original files (D1-05, D1-06)

Run once:

```
%run ./00_config
for f in FOLDERS:
    dbutils.fs.mkdirs(f'{BASE}/{f}')
    dbutils.fs.mkdirs(f'{ARCHIVE}/{f}')
dbutils.fs.mkdirs(f'{META}/schemas')
dbutils.fs.mkdirs(f'{META}/checkpoints')
```

Upload the original files **unchanged** into `source_archive/` (Catalog Explorer → your volume → open the folder → **Upload to this volume**, or the CLI):

Local file(s)	Upload to
`datasets/adls/primary_events/encounter_batch_01..12.csv`	`source_archive/primary_events/`
`datasets/adls/status_updates/status_updates.csv`	`source_archive/status_updates/`
`datasets/adls/secondary_events/secondary_events.csv`	`source_archive/secondary_events/`
`datasets/azure_sql_extracts/department_id.csv`	`source_archive/reference/department_id/`

Local file(s)	Upload to
`datasets/azure_sql_extracts/patient_id.csv`	`source_archive/reference/patient_id/`

CLI alternative for one folder:

```
databricks fs cp --recursive ./datasets/adls/primary_events
dbfs:/Volumes/<catalog>/hospital_ops/raw_landing/source_archive/primary_event
s
```

Integrity check – there must be 16 files in total (12 + 1 + 1 + 1 + 1) and the byte sizes must match your laptop copy:

```
for f in FOLDERS:
    for x in dbutils.fs.ls(f'{ARCHIVE}/{f}'):
        print(f, x.name, x.size)
```

Save the printed listing to `evidence/day1_archive_listing.txt` (this is D1-06).

S3.4 The release plan (how you will demonstrate incremental ingestion)

You will not copy everything into the landing folders at once. You release files in **waves** and run ingestion after each wave. Record the Bronze count after each wave in `evidence/day1_wave_log.md`.

Wave	Release	Purpose	Expected `bronze_encounter_events` rows after the run
W1	`encounter_batch_01.csv`	First micro-batch and lineage demo	1,000
W2	batches 02–09, plus status, secondary and both reference files	Normal incremental load	9,000 for encounters (520 / 310 / 110 / 150 for the others)

Wave	Release	Purpose	Expected `bronze_encounter_events` rows after the run
W3	batch 10	**Schema change**: expect one failure, then success	10,000
W4	batches 11–12	Finish	11,540

S4. Source profiling (70 min)

Why: profiling tells you what is in the data *before* you design cleaning rules. The guide expects counts, types, missing values, duplicates, extreme values, status distribution and key coverage. Create `notebooks/01_profile_sources.py` and save results to `docs/source_profile.md` (D1-07).

Profiling reads the **archive** copy, so it works before any Bronze table exists and never touches the landing folders.

```
# 01_profile_sources
%run ./00_config
from functools import reduce
from pyspark.sql import functions as F

def read_dir(sub):
    '''Read each CSV separately (batches 10-12 have an extra column) and
union by name.'''
    files = sorted([f.path for f in dbutils.fs.ls(f'{ARCHIVE}/{sub}') if
f.name.endswith('.csv')])
    dfs = [spark.read.option('header', 'true').csv(p).withColumn('_file',
F.lit(p.split('/')[-1])) for p in files]
    return reduce(lambda a, b: a.unionByName(b, allowMissingColumns=True),
dfs)

prim = read_dir('primary_events')
cdc  = read_dir('status_updates')
sec  = read_dir('secondary_events')
dept = read_dir('reference/department_id')
pat  = read_dir('reference/patient_id')
```

> **Why per-file reads?** If you read a whole folder with one `spark.read`, Spark takes column names from one header and applies them to every file. The extra column in batches 10–12 would be lost or misaligned. This is exactly the kind of trap Bronze must avoid.

P1. Row counts and files

```
for name, df in [('primary', prim), ('cdc', cdc), ('secondary', sec),
('department', dept), ('patient', pat)]:
    print(name, df.count())
prim.groupBy('_file').count().orderBy('_file').show(20, False)
print('distinct encounter_id:',
prim.select('encounter_id').distinct().count())
print('surplus rows:', prim.count() -
prim.select('encounter_id').distinct().count())
```

P2. Column types and schema drift

```
prim.printSchema()                    # everything is string (CSV without
inference)
prim.groupBy('_file').agg(F.count('source_schema_version').alias('rows_with_v
ersion')).orderBy('_file').show(20, False)
```

P3. Missing values

```
prim.select([F.sum(F.col(c).isNull().cast('int')).alias(c) for c in
prim.columns]).show(vertical=True)
```

P4. Duplicates – classify them, do not just count them

```
base12 = REQUIRED_PRIMARY
dup = (prim.groupBy('encounter_id')
       .agg(F.count('*').alias('n_rows'),
            F.countDistinct('_file').alias('n_files'),

F.size(F.collect_set(F.struct(*base12))).alias('n_variants_on_12_cols'),
            F.size(F.collect_set('ingest_timestamp')).alias('n_ingest_ts'))
       .filter('n_rows > 1'))
dup.groupBy('n_rows', 'n_files', 'n_variants_on_12_cols',
'n_ingest_ts').count().show()
```

P5. Extreme values

```
b =
prim.dropDuplicates(base12).select(F.col('bill_amount').cast('double').alias(
'b'))
q1, q3 = b.stat.approxQuantile('b', [0.25, 0.75], 0.0)
iqr = q3 - q1
for k in [1.5, 3, 5]:
    print('k =', k, 'threshold =', round(q3 + k * iqr, 2), 'rows above =',
b.filter(F.col('b') > q3 + k * iqr).count())
b.orderBy(F.col('b').desc()).show(40)
prim.select(F.min(F.col('wait_minutes').cast('int')),
F.max(F.col('wait_minutes').cast('int'))).show()
```

P6. Categories, timestamps, key coverage

```
for c in ['visit_type', 'encounter_status', 'priority', 'source_system']:
    prim.groupBy(c).count().orderBy(F.desc('count')).show()
prim.select(F.min('event_timestamp'), F.max('event_timestamp')).show()
print('event after ingest:',
prim.filter(F.col('event_timestamp').cast('timestamp') >
F.col('ingest_timestamp').cast('timestamp')).count())
print('dept keys unmatched:', prim.join(dept, 'department_id',
'left_anti').count())
print('patient keys unmatched:', prim.join(pat, 'patient_id',
'left_anti').count())
cdc.groupBy('previous_status',
'new_status').count().orderBy('previous_status', 'new_status').show(20)
print('cdc ids not in primary:',
cdc.join(prim.select('encounter_id').distinct(), 'encounter_id',
'left_anti').count())
print('secondary ids not in primary:',
sec.join(prim.select('encounter_id').distinct(), 'encounter_id',
'left_anti').count())
print('cdc sequence unique:',
cdc.select('source_sequence').distinct().count(), 'of', cdc.count())
sec.groupBy('event_type').count().show()
```

What you should find (my pre-check of the provided files)

Re-verify every line on your run. A difference is a **finding** to log, not something to hide.

Check	Expected result
Primary rows / files	11,540 rows in 12 files (batches 01–11 have 1,000 each; batch 12 has 540)
Distinct `encounter_id`	11,500, so 40 surplus rows (TC01)
Duplicate shape	Each of the 40 repeated IDs appears exactly twice, with **identical values on all 12 base columns including the same `ingest_timestamp`**. 37 pairs span two files; 3 pairs sit inside one file.

Check	Expected result
Why a naive duplicate check says 28 or 40	12 of the pairs have one copy in batches 01–09 and the other in batches 10–12; the later copy carries `source_schema_version = v2` and the earlier does not. Compared on all 13 columns they look different; on the 12 base columns they are identical.
Missing values	12 nulls, all in `visit_type`; no other nulls in the 12 base columns
Schema drift	`source_schema_version` present only in batches 10–12 (2,540 rows); null for the other 9,000
Extreme values	In `bill_amount`. Typical median about 780; maximum about 111,535. A rule of Q3 + 3×IQR (about 3,332) isolates exactly **35** rows; the 35th largest value is about 4,967 and the next is about 1,500, a clear gap. (A fixed 5,000 threshold gives only 34, so do not use 5,000.)
`wait_minutes`	Range 2–160, no negatives or extremes
`priority`	Contains only "USD" in every row – looks like a currency value in the wrong column (DEC-11)
Categories	`visit_type`: INPATIENT, OUTPATIENT, EMERGENCY (+12 null). `encounter_status`: OPEN, PENDING, COMPLETED, FAILED. `source_system`: ADLS_EVENT
Timestamps	`event_timestamp` from 2026-07-01 to 2026-08-01; none later than `ingest_timestamp`
Key coverage	All `department_id` and `patient_id` values exist in the reference extracts (110 and 150 rows)
Status CDC	520 rows; `source_sequence` 1–520 all unique; new status is CANCELLED, REVERSED or COMPLETED; all dated 2026-08-10; every ID exists in primary; every `previous_status` equals the primary status

Check	Expected result
Secondary events	310 rows; types ALERT 98, REVIEW 118, CHECK 94; every `encounter_id` exists in primary

Write these into `docs/source_profile.md` as: **Finding → Evidence (query + output) → Classification (valid / invalid / valid-but-flagged) → Proposed handling → Decision ID**. Update DEC-08 to DEC-12 with what the data showed. Update TC01 and TC02 evidence now.

S5. Design documents (70 min)

Commit all five documents to `docs/` on `feature/day1-docs`. They are the contract that Days 2–5 build against, so be specific.

S5.1 High-level design – HLD (D1-08, `docs/02_hld.md`)

One diagram plus one table. Keep the diagram as text (Mermaid) so Git can version it; most Git providers render it.

```
flowchart LR
  A[ADLS Gen2 CSV files] -->|Auto Loader| B[(Bronze Delta)]
  S[Azure SQL or CSV extract] -->|JDBC or staged extract| B
  B -->|quality, dedup, CDC MERGE| C[(Silver Delta)]
  C -->|facts, dimensions, KPIs| D[(Gold Delta)]
  D --> E[SQL Warehouse]
  E --> F[AI/BI Dashboard and Genie]
  D --> G[MLflow baseline]
  W[Workflows] -. orchestrates .-> B
  W -.-> C
  W -.-> D
  U[Unity Catalog] -. governs .-> B
  U -.-> C
  U -.-> D
  R[Git folders] -. holds code and runbook .-> W
```

Component	Role in this project	Built
ADLS Gen2 + external volume	Holds raw files; governed by Unity Catalog	Day 1

Component	Role in this project	Built
Auto Loader	Loads each new file exactly once, remembers progress	Day 1
Azure SQL (simulated by CSV)	Reference entities: department and patient	Day 1
Bronze Delta	Raw, append-only, full lineage	Day 1
Silver Delta	Typed, deduplicated, quality-checked, current status	Day 2
Gold Delta	Facts, dimensions and KPI marts	Day 3
Workflows	Runs the pipeline in order with retries and alerts	Day 4
Unity Catalog	Permissions, lineage, masking	Days 1–4
SQL Warehouse, AI/BI, Genie, MLflow	Consumption and analytics extension	Day 5
Git folders	Code, tests, runbook, PR history	Day 1 onward

S5.2 Low-level design – LLD (D1-09, `docs/03_lld.md`)

One row per notebook. Day 1 notebooks are real; later ones are **reserved names** so other documents can refer to them.

Notebook	Input	Output	Key logic	Day
`00_config`	widget `catalog`	constants, `release()`	single source of paths and names	1
`01_profile_sources`	`source_archive/`	`docs/source_profile.md`	counts, nulls, duplicates, extremes, key coverage	1
`02_bronze_lib`	landing folders	functions	Auto Loader + `foreachBatch` append with lineage	1
`03_bronze_run`	`02_bronze_lib`	five Bronze tables	wave releases and runs	1
`04_bronze_audit`	Bronze + archive	`ops_bronze_audit`	expected vs actual counts per table and per file	1

Notebook	Input	Output	Key logic	Day
`tests/test_day1_bronze`	Bronze + audit	`ops_test_results`	contract, lineage, replay tests	1
`10_silver_events`	`bronze_encounter_events`	`silver_encounter_events`, `silver_rejected_records`	casting, DQ rules, dedup	2 (reserved)
`11_silver_cdc`	`bronze_status_cdc`	`silver_status_history`, `silver_encounter_current`	ordered MERGE, late-event guard	2 (reserved)
`12_silver_masking`	Silver, reference	masked views / column masks	restrict identifiers	2 (reserved)
`20_gold_build`	Silver	facts, dimensions, `mart_encounter_daily`	KPIs, incremental MERGE	3 (reserved)
`21_gold_validate`	Silver, Gold	reconciliation results	expected vs actual	3 (reserved)
`30_workflow`	all of the above	Workflows job	DAG, retries, alerts	4 (reserved)

S5.3 Entity relationship diagram – ERD (D1-10, `docs/04_erd.md`)

Logical model of the sources (Day 1), with Silver and Gold tables listed beneath it as **proposals** to confirm on Days 2 and 3.

```
erDiagram
  DEPARTMENT ||--o{ ENCOUNTER_EVENT : hosts
  PATIENT ||--o{ ENCOUNTER_EVENT : has
  ENCOUNTER_EVENT ||--o{ STATUS_CHANGE : changes
  ENCOUNTER_EVENT ||--o{ SECONDARY_EVENT : triggers
  ENCOUNTER_EVENT {
    string encounter_id
    string department_id FK
    string patient_id FK
    string encounter_status
    string doctor_id
  }
  STATUS_CHANGE {
    string encounter_id FK
    int source_sequence
    string new_status
  }
  SECONDARY_EVENT {
    string event_id
    string encounter_id FK
```

```
    string event_type
  }
  DEPARTMENT {
    string department_id PK
  }
  PATIENT {
    string patient_id PK
  }
```

Planned table	Grain (one row per…)	Business key
`silver_encounter_events`	encounter after deduplication	`encounter_id`
`silver_encounter_current`	encounter (latest valid status)	`encounter_id`
`silver_status_history`	valid status change	`encounter_id` + `source_sequence`
`silver_rejected_records`	rejected source row per rule	source file + row number + rule ID
`dim_department`, `dim_patient`	reference key	`department_id` / `patient_id`
`fact_encounter_activity`	encounter	`encounter_id`
`fact_status_transitions`	status change	`encounter_id` + `source_sequence`
`mart_encounter_daily`	event date per department (proposed)	`event_date` + `department_id`

S5.4 Source-to-target mapping (D1-11, `docs/05_source_to_target_mapping.md`)

Bronze rule (final for Day 1): every source column maps to a Bronze column with the **same name**, stored as STRING, with no value changes. Lineage columns are added as below.

Bronze column	Filled from
`_source_file`	`_metadata.file_path` (full path)
`_source_file_name`	`_metadata.file_name`
`_source_file_mod_time`	`_metadata.file_modification_time`
`_source_row_number`	position of the row within its file (1 = first data row)
`_ingested_at`	`current_timestamp()` at load
`_batch_id`	Auto Loader micro-batch id
`_source_mode`	`ADLS_AUTOLOADER` or `CSV_EXTRACT_SIMULATION`
`_rescued_data`	filled by Auto Loader only if a value cannot be parsed

Silver typing (planned, confirm Day 2):

Source column	Silver type	Rule
`event_timestamp`, `ingest_timestamp`, `change_timestamp`	timestamp	unparseable → reject
`bill_amount`, `event_value`	decimal(12,2)	non-numeric or negative → reject; extreme → keep and flag
`wait_minutes`, `source_sequence`	int	non-numeric or negative → reject
`effective_date`	date	unparseable → reject
`visit_type`	string	missing → `UNKNOWN`, flagged
all keys and categories	string, trimmed, upper-cased where coded	keep raw copy alongside

S5.5 Data dictionary (D1-12, `docs/06_data_dictionary.md`)

Sensitivity: **Restricted** = identifier to be masked or access-controlled on Day 2. All other columns are Internal. The data is synthetic (A-01) but the practice is graded.

Primary encounter events (`bronze_encounter_events`)

Column	Meaning	Silver type	Allowed / observed	Sensitivity
`encounter_id`	Business key, format `ENCOUNTER-#######`	string	11,500 distinct	Internal
`event_timestamp`	When the event happened	timestamp	2026-07-01 to 2026-08-01	Internal
`ingest_timestamp`	When the source recorded it	timestamp	never before `event_timestamp`	Internal
`department_id`	Foreign key to department	string	110 valid keys	Internal
`patient_id`	Foreign key to patient	string	150 valid keys	**Restricted**
`visit_type`	Kind of visit	string	INPATIENT, OUTPATIENT, EMERGENCY; 12 missing	Internal
`encounter_status`	Status at event time	string	OPEN, PENDING, COMPLETED, FAILED	Internal

Column	Meaning	Silver type	Allowed / observed	Sensitivity
`priority`	Intended: priority level. Observed: only "USD" (DEC-11)	string	USD	Internal
`bill_amount`	Billed amount, USD assumed	decimal(12,2)	35 extreme values	Internal
`wait_minutes`	Minutes waited	int	2–160	Internal
`doctor_id`	Treating doctor, `DOCTOR_ID-####`	string	–	**Restricted**
`source_system`	Origin	string	ADLS_EVENT	Internal
`source_schema_version`	Contract version marker	string	`v2`, only in batches 10–12	Internal

Status changes (`bronze_status_cdc`)

Column	Meaning	Silver type	Allowed / observed
`encounter_id`	Encounter being changed	string	all exist in primary
`change_timestamp`	When the change applies	timestamp	all 2026-08-10
`previous_status`	Status before	string	OPEN, PENDING, COMPLETED, FAILED
`new_status`	Status after	string	CANCELLED, REVERSED, COMPLETED

Column	Meaning	Silver type	Allowed / observed
`change_reason`	Why	string	three values (e.g. OPERATIONS_UPDATE, CUSTOMER_ACTION)
`source_sequence`	Ordering number	int	1–520, unique
`source_system`	Origin	string	ADLS_CDC

Secondary events (`bronze_secondary_events`)

Column	Meaning	Silver type	Allowed / observed
`event_id`	Key, `SEC-######`	string	310 distinct
`encounter_id`	Foreign key to encounter	string	all exist in primary
`event_timestamp`	When it happened	timestamp	–
`related_entity_id`	Related department	string	`DEPARTMENT_ID-####`
`event_type`	Kind of event	string	ALERT, REVIEW, CHECK
`event_value`	Numeric reading	decimal	–
`source_system`	Origin	string	ADLS_AUX

Reference entities (`bronze_department_id` , `bronze_patient_id`) – both have `<key>` , `display_name` , `region` , `active_flag` (Y/N), `effective_date` , `risk_tier` . Department: 110 rows. Patient: 150 rows, and `display_name` is **Restricted**. Confirm the observed `region` and `risk_tier` values in profiling and write them in.

S6. Build Bronze with Auto Loader (100 min)

Switch to branch `feature/day1-bronze` (created from `main` , or from your docs branch if you prefer one PR).

S6.1 The concepts you need

- **Bronze rules:** append-only; no cleaning; no deleting duplicates; every row carries where it came from. If Silver is ever wrong you can rebuild it from Bronze, which is why Bronze must be faithful.
- **Auto Loader** watches a folder and loads each file once. It keeps two bookmarks: the **schema location** (what columns exist) and the **checkpoint** (which files are done). Delete neither casually.
- **`availableNow` trigger:** process everything new, then stop. This gives you batch-style runs with streaming safety, and it is what Workflows will call on Day 4.
- **`foreachBatch`:** lets us add the per-file row number (a window function, not allowed directly on a stream) and control the write.
- **Idempotent write (`txnAppId` + `txnVersion`):** if a run dies after writing but before the checkpoint is saved, the retry's write is recognised and skipped, so no duplicates.

S6.2 `notebooks/02_bronze_lib.py` (D1-14)

```
# Databricks notebook source
# 02_bronze_lib - Auto Loader functions for all five sources
%run ./00_config

# COMMAND ----------
from pyspark.sql import functions as F, Window

def contract_ok(columns):
    '''Primary feed contract: all 12 base columns must be present.'''
    missing = [c for c in REQUIRED_PRIMARY if c not in columns]
    return (len(missing) == 0, missing)

def run_stream(name):
    cfg = STREAMS[name]
    target = cfg['target']
    mode = cfg['mode']
    src = f'{BASE}/' + cfg['sub'] + '/'
    schema_loc = f'{META}/schemas/{name}'
    ckpt = f'{META}/checkpoints/{name}'
    app_id = f'bronze_{name}'

    raw = (spark.readStream.format('cloudFiles')
           .option('cloudFiles.format', 'csv')
```

```
            .option('header', 'true')
            .option('cloudFiles.inferColumnTypes', 'false')
            .option('cloudFiles.schemaLocation', schema_loc)
            .option('cloudFiles.schemaEvolutionMode', 'addNewColumns')
            .load(src))

    df = (raw
          .withColumn('_source_file', F.col('_metadata.file_path'))
          .withColumn('_source_file_name', F.col('_metadata.file_name'))
          .withColumn('_source_file_mod_time',
F.col('_metadata.file_modification_time'))
          .withColumn('_ingested_at', F.current_timestamp())
          .withColumn('_source_mode', F.lit(mode)))

    def write_batch(batch_df, batch_id):
        if name == 'encounters':
            ok, missing = contract_ok(batch_df.columns)
            if not ok:
                raise Exception(f'CONTRACT BREAK - missing columns:
{missing}')
        w = Window.partitionBy('_source_file').orderBy('_rid')
        out = (batch_df
               .withColumn('_rid', F.monotonically_increasing_id())
               .withColumn('_source_row_number', F.row_number().over(w))
               .withColumn('_batch_id', F.lit(int(batch_id)))
               .drop('_rid'))
        (out.write.format('delta').mode('append')
            .option('mergeSchema', 'true')
            .option('txnAppId', app_id)
            .option('txnVersion', int(batch_id))
            .saveAsTable(target))

    q = (df.writeStream
         .foreachBatch(write_batch)
         .option('checkpointLocation', ckpt)
         .trigger(availableNow=True)
         .start())
    q.awaitTermination()
    return q

def run_all(names=None):
    for n in (names or list(STREAMS)):
        print('running', n)
```

```
        run_stream(n)

def counts():
    rows = []
    for n, cfg in STREAMS.items():
        t = cfg['target']
        c = spark.table(t).count() if spark.catalog.tableExists(t) else 0
        rows.append((n, t, c))
    return spark.createDataFrame(rows, 'stream string, table_name string,
rows long')

def tag_tables():
    for n, cfg in STREAMS.items():
        t = cfg['target']
        m = cfg['mode']
        cmt = f'Bronze raw copy. Append-only. All columns STRING. Source
mode: {m}.'
        spark.sql(f'''ALTER TABLE {t} SET TBLPROPERTIES ('layer'='bronze',
'source_mode'='{m}')''')
        spark.sql(f'''COMMENT ON TABLE {t} IS '{cmt}' ''')

def reset_stream(name):
    '''DEV ONLY. Drops the table and deletes checkpoint + schema history for
one stream.
    Always also remove the released files you want re-loaded from the landing
folder, or
    simply re-release them. Never run this against a table other people
depend on.'''
    t = STREAMS[name]['target']
    spark.sql(f'DROP TABLE IF EXISTS {t}')
    dbutils.fs.rm(f'{META}/checkpoints/{name}', True)
    dbutils.fs.rm(f'{META}/schemas/{name}', True)
```

> **Warning about resets:** the idempotent write remembers batch numbers per `txnAppId`. If you delete a checkpoint but keep the table, the restarted stream reuses batch numbers and Delta will silently **skip** those writes. Always reset the table, checkpoint and schema folder **together** (`reset_stream`).

S6.3 `notebooks/03_bronze_run.py` – run the waves

Run each wave in its own cell and record the counts in `evidence/day1_wave_log.md`.

```
# 03_bronze_run
%run ./02_bronze_lib

# COMMAND ----------
# W1: first micro-batch (encounters only)
release('primary_events', ['encounter_batch_01.csv'])
run_stream('encounters')
display(counts())          # expect encounters = 1000

# COMMAND ----------
# W2: batches 02-09 plus the other four sources
release('primary_events', [f'encounter_batch_{i:02d}.csv' for i in range(2,
10)])
release('status_updates', ['status_updates.csv'])
release('secondary_events', ['secondary_events.csv'])
release('reference/department_id', ['department_id.csv'])
release('reference/patient_id', ['patient_id.csv'])
run_all()
display(counts())          # expect 9000 / 520 / 310 / 110 / 150

# COMMAND ----------
# W3: schema change. The first run is EXPECTED to fail once.
release('primary_events', ['encounter_batch_10.csv'])
try:
    run_stream('encounters')
except Exception as e:
    print('EXPECTED on first sight of the new column:', type(e).__name__)
    print(str(e)[:400])

# COMMAND ----------
run_stream('encounters')   # second run succeeds with the new column known
display(counts())          # expect encounters = 10000

# COMMAND ----------
# W4: finish
release('primary_events', ['encounter_batch_11.csv',
```

```
'encounter_batch_12.csv'])
run_stream('encounters')
tag_tables()
display(counts())          # expect 11540 / 520 / 310 / 110 / 150
```

What to capture as evidence (D1-13):

- The output of each `counts()` call
- The W3 error message (screenshot **and** copy the text into `evidence/day1_schema_change.md`) and the successful second run
- `DESCRIBE HISTORY <table>` for `bronze_encounter_events` showing one `WRITE` per micro-batch
- A short note that batches 10–12 are the only files with `source_schema_version`

S6.4 Quick checks right after W1

```
-- all raw values preserved, lineage present
SELECT * FROM hospital_ops.bronze_encounter_events LIMIT 5;

-- row 1 of batch 01 must be the first data line of the file
SELECT encounter_id, _source_file_name, _source_row_number, _ingested_at,
_batch_id
FROM hospital_ops.bronze_encounter_events
WHERE _source_file_name = 'encounter_batch_01.csv' AND _source_row_number IN
(1, 2)
ORDER BY _source_row_number;
-- expect ENCOUNTER-0003958 then ENCOUNTER-0009377

DESCRIBE HISTORY hospital_ops.bronze_encounter_events;
```

If the first two IDs are different, row numbering is wrong: stop and fix it before continuing, because Day 2 rejected-record traceability depends on it.

S6.5 Azure SQL reference data (D1-13, part 2)

- **Default path (expected):** the two reference CSVs are loaded through Auto Loader (already done in W2). Their `_source_mode` is `CSV_EXTRACT_SIMULATION`. State this in the decision log (DEC-03), the data dictionary and the handover deck. **Never** write "loaded from Azure SQL".

- **If a live, approved connection exists:** import the CSVs into Azure SQL using `technical_reference/azure_sql_ddl.sql` (it creates `dbo.department_id` and `dbo.patient_id`), then read through JDBC with secrets, and label that table `AZURE_SQL_JDBC`. Only claim this if the run actually succeeded.

```
# ONLY with an approved connection. Otherwise stay on the simulation.
jdbc_url = dbutils.secrets.get('<scope>', 'azuresql-jdbc-url')
user = dbutils.secrets.get('<scope>', 'azuresql-user')
pwd = dbutils.secrets.get('<scope>', 'azuresql-password')
df = (spark.read.format('jdbc')
      .option('url', jdbc_url).option('dbtable', 'dbo.department_id')
      .option('user', user).option('password', pwd).load())
```

A JDBC read is a full snapshot every time, so append it with an `_ingested_at` timestamp and let Silver keep only the latest snapshot. Never put the URL or password in code or in Git.

S7. Bronze audit and lineage (35 min)

S7.1 Reconcile Bronze to the source files (D1-15)

Run this **after W4**. It compares, for every file, the row count read directly from the archive with the rows in Bronze, then compares table totals with fixed expected numbers. Results are stored in `ops_bronze_audit` so you have history.

```
# 04_bronze_audit
%run ./00_config
import datetime
from pyspark.sql import functions as F

run_ts = datetime.datetime.now()
rows = []

def add(check, expected, actual):
    status = 'PASS' if str(expected) == str(actual) else 'FAIL'
    rows.append((run_ts, check, str(expected), str(actual), status))

# 1) per file: archive rows vs Bronze rows
for key, cfg in STREAMS.items():
    target = cfg['target']
```

```
    by_file = {r['_source_file_name']: r['n'] for r in

spark.table(target).groupBy('_source_file_name').agg(F.count('*').alias('n'))
.collect()}
    total_src = 0
    for f in dbutils.fs.ls(f'{ARCHIVE}/' + cfg['sub']):
        if f.name.endswith('.csv'):
            n_src = spark.read.option('header', 'true').csv(f.path).count()
            total_src += n_src
            add(f'{target} :: {f.name} rows', n_src, by_file.get(f.name, 0))
    add(f'{target} total rows', total_src, spark.table(target).count())

# 2) fixed expectations from the project manifest
MANIFEST = {'bronze_encounter_events': 11540, 'bronze_status_cdc': 520,
            'bronze_secondary_events': 310, 'bronze_department_id': 110,
'bronze_patient_id': 150}
for t, n in MANIFEST.items():
    add(f'manifest :: {t}', n, spark.table(f'{FQ}.{t}').count())

# 3) duplicates are kept in Bronze (surplus rows must still be 40)
enc = spark.table(STREAMS['encounters']['target'])
add('encounters surplus rows (kept, not removed)', 40, enc.count() -
enc.select('encounter_id').distinct().count())

aud = spark.createDataFrame(rows, 'run_ts timestamp, check_name string,
expected string, actual string, status string')
aud.write.mode('append').saveAsTable(f'{FQ}.ops_bronze_audit')
display(aud)
print('FAILS:', aud.filter("status = 'FAIL'").count())
```

Every line must be PASS. A FAIL means data was lost, duplicated or loaded early – investigate before moving on. Export the displayed table as CSV into `evidence/day1_bronze_audit.csv`.

S7.2 Show lineage (D1-16)

1. **Catalog Explorer** → `hospital_ops` → `bronze_encounter_events` → **Lineage** tab → screenshot. It should show the source files/volume path feeding the table.
2. Query the system table (needs system tables enabled; lineage can lag a few minutes):

```
SELECT source_table_full_name, source_path, target_table_full_name,
event_time
FROM system.access.table_lineage
WHERE target_table_full_name LIKE '%hospital_ops.bronze_%'
ORDER BY event_time DESC;
```

If system tables are not available to you, say so in the RAID tracker and use Delta history plus the `_source_file` / `_source_row_number` columns as the lineage evidence (they trace every row to its file and line).

S7.3 Start the single-encounter trace for the Day 5 walkthrough

The client acceptance meeting asks you to follow one `encounter_id` from source to Gold. Start now with **ENCOUNTER-0000001**, which appears in all three event sources. Save the output in `evidence/day1_trace_ENCOUNTER-0000001.md`; extend it on Days 2 and 3.

```
SELECT 'primary' AS src, encounter_id, encounter_status AS status_or_type,
_source_file_name, _source_row_number
FROM hospital_ops.bronze_encounter_events WHERE encounter_id = 'ENCOUNTER-
0000001'
UNION ALL
SELECT 'status_cdc', encounter_id, new_status, _source_file_name,
_source_row_number
FROM hospital_ops.bronze_status_cdc WHERE encounter_id = 'ENCOUNTER-0000001'
UNION ALL
SELECT 'secondary', encounter_id, event_type, _source_file_name,
_source_row_number
FROM hospital_ops.bronze_secondary_events WHERE encounter_id = 'ENCOUNTER-
0000001';
```

Expected: one primary row in `encounter_batch_02.csv` (status PENDING, OUTPATIENT, department 0001, patient 0001), one status change PENDING to CANCELLED with sequence 1 from `status_updates.csv`, and one ALERT event (`SEC-000001`) from `secondary_events.csv`.

S7.4 Enforce append-only (do this last, after S8 passes)

```
ALTER TABLE hospital_ops.bronze_encounter_events SET TBLPROPERTIES
('delta.appendOnly' = 'true');
-- repeat for the other four Bronze tables
```

This makes Delta reject UPDATE, DELETE and MERGE on those tables, which proves the "raw history is immutable" requirement. If you ever need a clean restart you drop the table with `reset_stream`.

S8. Day 1 tests (50 min)

Create `tests/test_day1_bronze.py`. It collects every result (so one failure does not hide the others), stores them in `ops_test_results`, and fails the notebook if anything failed.

```
# Databricks notebook source
# tests/test_day1_bronze
%run ../notebooks/02_bronze_lib

# COMMAND ----------
import datetime
from pyspark.sql import functions as F

run_ts = datetime.datetime.now()
results = []

def check(test_id, name, condition, detail=''):
    status = 'PASS' if condition else 'FAIL'
    results.append((run_ts, test_id, name, status, str(detail)))
    print(status, test_id, name, detail)

def T(n):
    return spark.table(STREAMS[n]['target'])

enc = T('encounters')
dp = T('department_id')
pa = T('patient_id')

# COMMAND ----------
# TC01 counts
n = enc.count()
```

```
d = enc.select('encounter_id').distinct().count()
check('TC01', 'rows 11540 / distinct 11500 / surplus 40', (n, d, n - d) ==
(11540, 11500, 40), (n, d, n - d))

# TC02 missing and extreme values
nulls = enc.filter(F.col('visit_type').isNull()).count()
check('TC02', 'visit_type nulls = 12', nulls == 12, nulls)
u =
enc.dropDuplicates(REQUIRED_PRIMARY).select(F.col('bill_amount').cast('double
').alias('b'))
q1, q3 = u.stat.approxQuantile('b', [0.25, 0.75], 0.0)
thr = q3 + 3 * (q3 - q1)
ext = u.filter(F.col('b') > thr).count()
check('TC02', 'extreme bill_amount (Q3 + 3*IQR) = 35', ext == 35, (round(thr,
2), ext))

# D1-T01 source contract
ok, missing = contract_ok(enc.columns)
check('D1-T01', 'all 12 base columns present', ok, missing)
allowed = set(REQUIRED_PRIMARY + OPTIONAL_PRIMARY + LINEAGE_COLS)
extra = sorted(set(enc.columns) - allowed)
check('D1-T01', 'no unexpected columns', len(extra) == 0, extra)
check('D1-T01', 'contract check rejects a broken feed',
contract_ok(['encounter_id'])[0] is False)

# D1-T02 lineage columns populated on every table
must = ['_source_file', '_source_file_name', '_source_row_number',
'_ingested_at', '_batch_id', '_source_mode']
for k, cfg in STREAMS.items():
    bad = spark.table(cfg['target']).filter(' OR '.join([f'{c} IS NULL' for c
in must])).count()
    check('D1-T02', f'{k}: lineage columns populated', bad == 0, bad)

# D1-T03 row numbering follows file order
first = (enc.filter((F.col('_source_file_name') == 'encounter_batch_01.csv')
& (F.col('_source_row_number') <= 2))
         .orderBy('_source_row_number').select('encounter_id').collect())
ids = [r[0] for r in first]
check('D1-T03', 'rows 1-2 of batch 01 are the first data lines', ids ==
['ENCOUNTER-0003958', 'ENCOUNTER-0009377'], ids)
g = enc.groupBy('_source_file_name').agg(F.count('*').alias('n'),
F.max('_source_row_number').alias('mx'),
```

```
F.countDistinct('_source_row_number').alias('dn'))
check('D1-T03', 'row numbers run 1..n in every file', g.filter('n != mx OR n
!= dn').count() == 0)

# D1-T04 schema change handled
v = enc.filter(F.col('source_schema_version').isNotNull())
vf = sorted([r[0] for r in
v.select('_source_file_name').distinct().collect()])
check('D1-T04', 'source_schema_version only in batches 10-12 (2540 rows)',
      (v.count(), vf) == (2540, ['encounter_batch_10.csv',
'encounter_batch_11.csv', 'encounter_batch_12.csv']), (v.count(), vf))

# D1-T05 reference simulation
check('D1-T05', 'reference rows 110 / 150', (dp.count(), pa.count()) == (110,
150))
check('D1-T05', 'reference keys unique',
dp.select('department_id').distinct().count() == 110 and
pa.select('patient_id').distinct().count() == 150)
check('D1-T05', 'reference labelled as simulation',
dp.select('_source_mode').distinct().collect()[0][0] ==
'CSV_EXTRACT_SIMULATION')

# TC07 (Bronze level) key coverage
ud = enc.join(dp, 'department_id', 'left_anti').count()
up = enc.join(pa, 'patient_id', 'left_anti').count()
check('TC07', 'no unmatched department / patient keys', ud == 0 and up == 0,
(ud, up))

# D1-T06 latest audit has no failure
audit = spark.table(f'{FQ}.ops_bronze_audit')
last = audit.agg(F.max('run_ts')).collect()[0][0]
fails = audit.filter((F.col('run_ts') == last) & (F.col('status') ==
'FAIL')).count()
check('D1-T06', 'latest Bronze audit has no FAIL', fails == 0, fails)

# COMMAND ----------
# TC03 replay: running ingestion again must add nothing
before = {k: T(k).count() for k in STREAMS}
run_all()
after = {k: T(k).count() for k in STREAMS}
check('TC03', 'replay adds zero rows to every Bronze table', before == after,
(before, after))
```

```
# COMMAND ----------
res = spark.createDataFrame(results, 'run_ts timestamp, test_id string,
test_name string, status string, detail string')
res.write.mode('append').saveAsTable(f'{FQ}.ops_test_results')
display(res)
assert res.filter("status = 'FAIL'").count() == 0, 'Day 1 tests have failures
- read the table above'
```

Evidence: export the result table as `evidence/day1_test_log.csv` and paste the pass/fail summary into the worksheet.

Worksheet test	Day 1 status
TC01	Done and recorded
TC02	Counts recorded; the rules that handle them are Day 2
TC03	Done at Bronze level (re-checked at Gold on Day 3)
TC04, TC06, TC08, TC09, TC10	Later days
TC05	Contract check function exists and is unit-tested; the malformed-file run through the workflow is Day 4
TC07	Bronze-level key check done; the Gold join check is Day 3

S9. Commit, pull request and end-of-day log (20 min)

1. In the Git folder, review changes, write a message beginning `D1:` , **Commit & Push**.
2. Open pull requests: `feature/day1-docs` → `main` and `feature/day1-bronze` → `main` . Paste `docs/PR_CHECKLIST.md` into each description and tick it honestly.
3. Ask a reviewer (or, if working alone, self-review and write that in the PR). Merge only after the checklist is complete. If you cannot merge on Day 1, leave the PRs open and note it in the log.

4. Create `evidence/day1_summary.md` with: what was built; counts per wave; the audit result; the test result; surprises found in the data; open decisions (DEC-08 to DEC-12); tomorrow's first three tasks.
5. Update `decision_log.md` and `raid.md` one last time.

---

4. Deliverables register (Day 1)

ID	Deliverable	Location	Accepted when
D1-01	BRS summary and source grain	`docs/01_brs_summary.md`	All sections of S1.1 present
D1-02	Decision log	`docs/decision_log.md`	DEC-01 to DEC-07 closed, DEC-08 to DEC-12 open with Day 1 evidence
D1-03	RAID tracker	`docs/raid.md`	R-01 to R-05, A-01, D-01 present with owners
D1-04	Git repo, branches, PR checklist	repository	Skeleton pushed, `main` protected, checklist committed
D1-05	Schema and external volume	Unity Catalog	`DESCRIBE VOLUME` works; schema `hospital_ops` exists
D1-06	Staged original files	`source_archive/` + `evidence/day1_archive_listing.txt`	16 files, sizes equal your local copies
D1-07	Source profile	`docs/source_profile.md`	Every table in S4 verified with query evidence
D1-08	HLD	`docs/02_hld.md`	Diagram and component table committed

ID	Deliverable	Location	Accepted when
D1-09	LLD	`docs/03_lld.md`	Every notebook listed with input, output, logic
D1-10	ERD	`docs/04_erd.md`	Source ERD plus planned table grains
D1-11	Source-to-target mapping	`docs/05_source_to_target_mapping.md`	Bronze rule, lineage columns and Silver typing present
D1-12	Data dictionary	`docs/06_data_dictionary.md`	All five sources documented with sensitivity
D1-13	Five Bronze tables	`hospital_ops.bronze_*`	Counts 11,540 / 520 / 310 / 110 / 150; wave log and schema-change evidence saved
D1-14	Ingestion notebooks	`notebooks/00_config`, `02_bronze_lib`, `03_bronze_run`	Run top to bottom without edits
D1-15	Bronze audit	`ops_bronze_audit`, `evidence/day1_bronze_audit.csv`	Latest run has no FAIL
D1-16	Lineage evidence	screenshot + query output	Shows source path to Bronze table; encounter trace started
D1-17	Day 1 test log	`ops_test_results`, `evidence/day1_test_log.csv`	All checks PASS
D1-18	Pull requests	Git provider	Both PRs open or merged with checklist ticked

---

5. Hand-off contract to Day 2 (X-xx)

Day 2 may rely on the following and nothing else. If any item is not true, Day 2 must not start.

ID	Guarantee
X-01	Silver reads **only from the five Bronze tables**, never from files or the archive.
X-02	Bronze tables exist under `<catalog>.hospital_ops` with the names in C-08 and are append-only.
X-03	All source columns are STRING and unmodified. Primary has 12 base columns plus optional `source_schema_version` (null for batches 01–09).
X-04	Every row has `_source_file`, `_source_file_name`, `_source_row_number` (1-based position in file) and `_ingested_at`. `silver_rejected_records` must reuse these for traceability.
X-05	Duplicates are still present in Bronze (11,540 rows, 11,500 distinct IDs). All 40 pairs are identical on the 12 base columns and share the same `ingest_timestamp`, so Day 2 needs a documented tie-breaker (e.g. latest `_source_file_name`, then `_source_row_number`).
X-06	The 35 extreme `bill_amount` values are isolated by Q3 + 3×IQR; they are flagged, not rejected, unless Day 2 decides otherwise (DEC-09).
X-07	The status file has 520 rows, all dated 2026-08-10, with unique `source_sequence`. It contains no late or out-of-order events, so Day 2 must author its own test files for TC04 (under `tests/data/`).
X-08	Reference tables are a **CSV simulation** (`_source_mode = CSV_EXTRACT_SIMULATION`); any document that mentions them must say so.

ID	Guarantee
X-09	`00_config` is the only place that holds the catalog, paths and stream definitions; Day 2 notebooks begin with `%run ./00_config`.
X-10	`ops_bronze_audit` and `ops_test_results` exist and are reused by later days; keep their column layout.
X-11	Open decisions carried into Day 2: DEC-08 tie-breaker, DEC-09 outlier rule, DEC-10 status mapping, DEC-11 `priority`, plus the masking approach for the Restricted columns. DEC-12 (bed capacity) is needed on Day 3.
X-12	Walkthrough encounter for the client demo: `ENCOUNTER-0000001`.

---

6. Troubleshooting

Symptom	Likely cause	Fix
`PERMISSION_DENIED` creating the volume	Missing `CREATE EXTERNAL VOLUME` on the external location	Ask the admin; or use a managed volume and log DEC-13
Auto Loader runs but loads 0 rows	Files are still only in `source_archive/`	Run `release(...)` and confirm with `dbutils.fs.ls` on the landing folder
Failure on batch 10 mentioning an unknown or new field	Expected schema change (`addNewColumns`)	Run the stream again; log it as evidence
Second run still fails	`mergeSchema` missing, or schema folder deleted	Confirm the option in `write_batch`; do not delete `_meta/schemas`

Symptom	Likely cause	Fix
Rows missing after a reset	Checkpoint deleted but table kept, so idempotent writes were skipped	Use `reset_stream` (table + checkpoint + schema together), re-release files, rerun
Bronze contains rows from the archive	A stream points at `source_archive`	Check `STREAMS[...]['sub']` in `00_config`
Row numbers not in file order	`_rid` ordering was removed or the file was split across partitions	Restore the `_rid` window logic from S6.2
Table not found	`catalog` widget still holds the placeholder	Set the widget and re-run `00_config`
Column name looks odd (trailing space or carriage return)	Source formatting	Print `[repr(c) for c in enc.columns]`, log it as a finding, do not rename in Bronze
Lineage tab or `system.access.table_lineage` is empty	Lineage lag or system tables not enabled	Wait a few minutes; otherwise rely on `_source_file` evidence and note it in RAID

---

7. Day 1 sign-off checklist

- [ ] `DESCRIBE VOLUME` works and all 16 original files are in `source_archive/`
- [ ] Four waves were run and the counts match the wave table (1,000 / 9,000 / 10,000 / 11,540 for encounters)
- [ ] The batch 10 schema-change failure and recovery are saved as evidence
- [ ] Five Bronze tables exist, tagged as `layer = bronze`, with the correct `source_mode`
- [ ] `ops_bronze_audit` latest run: zero FAIL
- [ ] `test_day1_bronze` run: zero FAIL, including the replay test
- [ ] Bronze raw values untouched; duplicates still present
- [ ] Lineage screenshot and `ENCOUNTER-0000001` trace saved

- [ ] HLD, LLD, ERD, mapping, data dictionary and profile committed
- [ ] Decision log and RAID updated; DEC-08 to DEC-12 open with evidence
- [ ] Pull requests opened with the checklist ticked
- [ ] No tokens, passwords or hard-coded catalog names in Git